3 함수

MATLAB Programming

Jin Hee Yoon
Dept. of Artificial Intelligence and Information Technology
Sejong University

Spring, 2026

함수에 대한 도움말 얻기

응용에 적절한 함수를 찾기 위해 `lookfor` 명령을 사용할 수 있다.

예를 들어, 허수를 다루는 함수의 목록을 얻기 위하여 `lookfor imaginary`를 입력한다. 결과를 보면 다음과 같다.

```
>> lookfor imaginary

imag    Complex imaginary part
i       Imaginary unit
j       Imaginary unit
```

일반적인 수학 함수들

지수함수

`exp(x)` 지수함수; e^x

`sqrt(x)` 제곱근; $\sqrt{x}$

로그함수

`log(x)` 자연로그; $\ln x$

`log10(x)` 상용 로그(밑수 10); $\log_{10} x$

일반적인 수학 함수들

복소수

`abs(x)` 절대값

`angle(x)` 복소수의 각도

`conj(x)` 켤레 복소수

`imag(x)` 복소수의 허수부

`real(x)` 복소수의 실수부

일반적인 수학 함수들

수치

`ceil(x)` $+\infty$에 가까운 정수로 올림

`fix(x)` 0에 가까운 정수로 내림

`floor(x)` $-\infty$에 가까운 정수로 내림

`round(x)` 가장 가까운 정수로 반올림

`sign(x)` Signum 함수

$$\mathrm{sign}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0 \end{cases}$$

복소수 연산-1

```
>> x = -3 + 4i;
>> y = 6 - 8i;

>> mag_x = abs(x)
mag_x =
    5.0000

>> mag_y = abs(y)
mag_y =
   10.0000

>> mag_product = abs(x*y)
mag_product =
   50.0000
```

복소수 연산-2

```
>> angle_x = angle(x)
angle_x =
    2.2143

>> angle_y = angle(y)
angle_y =
   -0.9273

>> sum_angles = angle_x + angle_y
sum_angles =
    1.2870

>> angle_product = angle(x*y)
angle_product =
    1.2870
```

배열 연산

MATLAB의 장점 중의 하나는 배열을 자동적으로 하나의 변수 취급한다는 것이다. 예를 들어, $5, 7, 15$의 제곱근을 계산하기 위하여 다음과 같이 입력한다.

```
>> x = [5,7,15];
>> y = sqrt(x)
y =
    2.2361    2.6458    3.8730
```

함수 입력변수 표현-1

- MATLAB에서는 함수 입력에 반드시 **괄호()** 사용한다.

  `sin 2` $\rightarrow$ `sin(2)`

- 배열 원소를 사용할 때도 동일하다.

  `sin [x(2)]` $\rightarrow$ `sin(x(2))`

- 즉, **함수명 뒤에는 항상 소괄호를 이용 (입력변수)**를 써야한다

함수 입력변수 표현-2

- 수식은 MATLAB 문법에 맞게 입력
  - $\sin(x^2 + 5) \quad \rightarrow \quad$ `sin(x.^2 + 5)`
  - $\sin(\sqrt{x} + 1) \quad \rightarrow \quad$ `sin(sqrt(x) + 1)`
- 함수 안에 또 다른 함수 사용 가능
- 괄호와 연산자 우선순위에 주의

함수 입력변수 표현-3

- $\sin^2 x$ 는 $(\sin x)^2$ 의미
- MATLAB 표현
  - `(sin(x))^2`
- 잘못된 표현
  - `sin^2(x)`, `sin^2 x`, `sin(x^2)`

함수 입력변수 표현-4

- MATLAB 삼각함수는 **radian** 기준
  - `sin(5)` = 5 radian의 사인
- degree → radian 변환식을 사용하여 radian 입력해야 함에 주의한다.

$$\text{radians} = \text{degrees} \times \frac{\pi}{180}$$

삼각함수

`cos(x)`	코사인; $\cos x$
`cot(x)`	코탄젠트; $\cot x$
`csc(x)`	코시컨트; $\csc x$
`sec(x)`	시컨트; $\sec x$
`sin(x)`	사인; $\sin x$
`tan(x)`	탄젠트; $\tan x$

역 삼각함수

`acos(x)`	역 코사인; $\arccos x$
`acot(x)`	역 코탄젠트; $\operatorname{arccot} x$
`acsc(x)`	역 코시컨트; $\operatorname{arccsc} x$
`asec(x)`	역 시컨트; $\operatorname{arcsec} x$
`asin(x)`	역 사인; $\arcsin x$
`atan(x)`	역 탄젠트; $\arctan x$
`atan2(y,x)`	사분면을 고려한 역 탄젠트

하이퍼볼릭 함수

`cosh(x)`	하이퍼볼릭 코사인
`coth(x)`	하이퍼볼릭 코탄젠트
`csch(x)`	하이퍼볼릭 코시컨트
`sech(x)`	하이퍼볼릭 시컨트
`sinh(x)`	하이퍼볼릭 사인
`tanh(x)`	하이퍼볼릭 탄젠트

역 하이퍼볼릭 함수

`acosh(x)`	역 하이퍼볼릭 코사인
`acoth(x)`	역 하이퍼볼릭 코탄젠트
`acsch(x)`	역 하이퍼볼릭 코시컨트
`asech(x)`	역 하이퍼볼릭 시컨트
`asinh(x)`	역 하이퍼볼릭 사인
`atanh(x)`	역 하이퍼볼릭 탄젠트

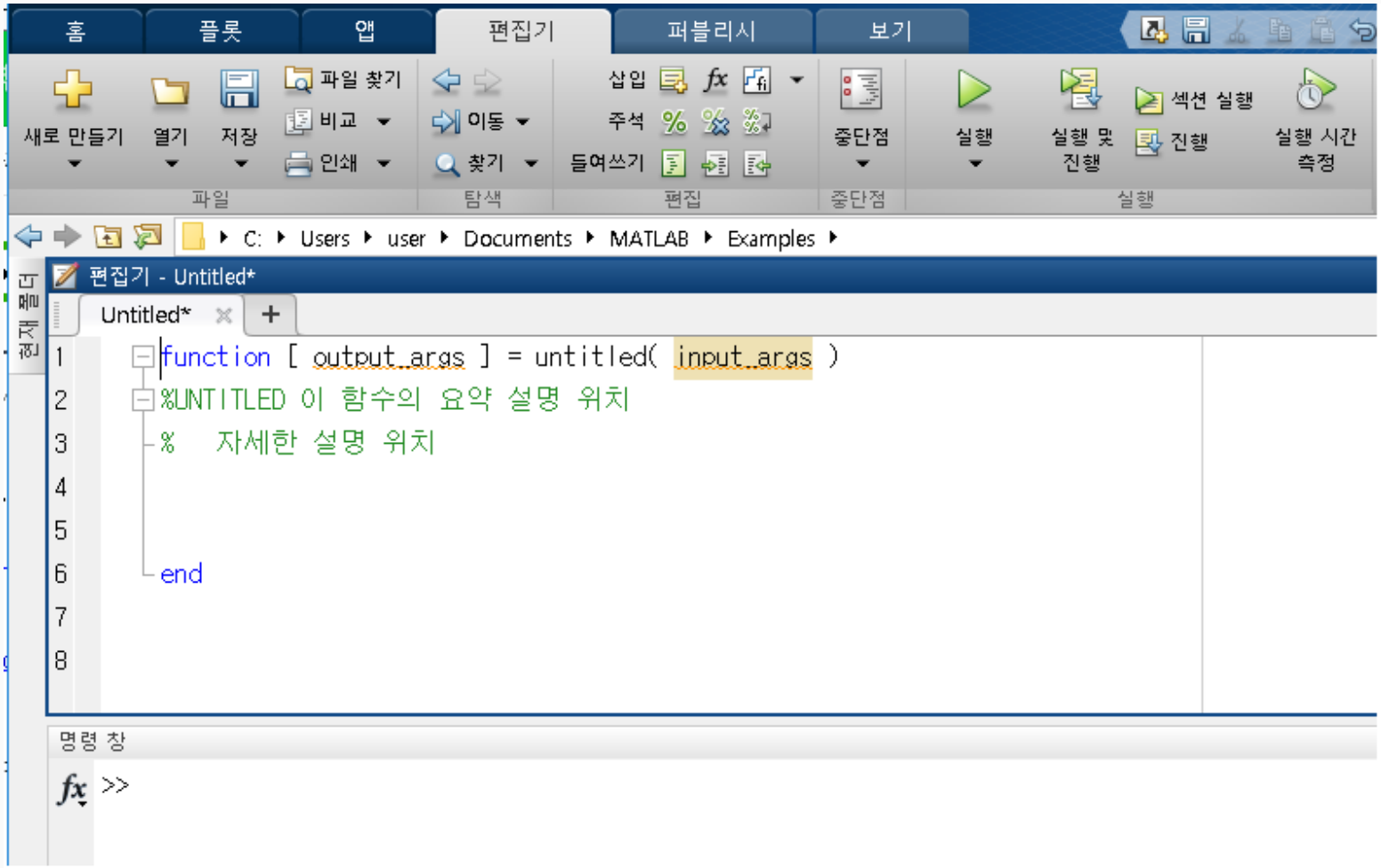
사용자 정의 함수-1

- 함수 파일 생성: **New** → **Function** 선택한다. (Script 도 가능)
- 함수 파일은 **함수 정의문**으로 시작해야 한다.
- 함수 정의문이 함수 M-파일과 스크립트 M-파일을 구분하는 기준이 된다.

사용자 정의 함수-2

- 함수 정의 형식
  - `function [output] = name(input)`
- 출력: 대괄호 [ ]
- 입력: 소괄호 ( )
- 반드시 **함수 이름 = 파일 이름** 이 되어야 한다.

새로운 함수를 생성할 때의 디폴트 편집기 창-1



사용자 정의 함수: 예-1

```
function z = fun(x,y)
u = 3*x;
z = u + 6*y.^2;
```

- output 에 세미콜론 ; - 출력없이 계산만 한다.
- .^ - 입력이 벡터나 행렬인 경우를 위해 배열 연산을 많이 사용한다.

사용자 정의 함수: 예-2

이 함수를 출력 변수와 함께 호출하면

```
>> z = fun(3,7)
z =
   303
```

이 함수는 $x = 3$과 $y = 7$을 이용하여 z를 계산한다.

사용자 정의 함수: 예-3

이 함수를 출력 변수 없이 호출하면

```
>> fun(3,7)
ans =
   303
```

출력 변수를 따로 지정하지 않았으므로 결과는 `ans`에 저장된다. 따라서 `z`를 직접 확인하려 하면 MATLAB은 `Undefined function or variable 'z'.` 와 같은 에러를 출력한다.

사용자 정의 함수: 예-4

출력 변수를 또 다른 변수로 할당도 가능하다.

```
>> q = fun(3,7)
q =
   303
```

함수 호출 후에 세미콜론을 넣어 출력을 보이지 않게 할 수 있다. 예를 들어, `q = fun(3,7);`을 입력하면 값 q는 계산되지만 화면에 표시되지는 않는다.

로컬 변수-1

- 함수 내부에서 생성된 변수는 **로컬 변수**라고 헌다.
- 함수 밖(workspace)에서는 **사용 불가능**하다.
- 로컬 변수는 workspace 변수 리스트에도 나타나지 않는다.

함수 정의

```
function z = fun(x,y)
u = 3*x;
z = u + y;
```

로컬 변수-2

Command Window

```
>> x = 3; y = 7;
>> q = fun(x,y);
>> x
>> y
>> u
```

```
>> a = 3; b = 7;
>> q = fun(a,b);
>> a
>> b
>> u
```

- u는 함수 내부에서만 존재 → **로컬 변수** 이므로 에러가 발생한다.

입력 변수 순서

입력 변수의 순서가 중요하다. 함수의 입력부분 ()은 변수의 이름이 아닌 순서로 인식한다.

```
>> x = 7; y = 3;
>> z = fun(y,x)
z =
   303
```

두 번째 줄은 `z = fun(3,7)`와 동등하다.

입력으로서의 배열

배열을 입력 변수로 이용할 수 있다.

```
>> r = fun(2:4,7:9)
r =
   300   393   498
```

하나 이상의 출력

- 함수는 여러 개의 함수값 출력이 가능하다.
- 출력은 대괄호 [ ]로 표현한다.

다음 함수 circle 는 반경이 입력 변수로 주어졌을 때, 원의 면적 A와 원주 C를 계산한다.

```
function [A, C] = circle(r)
A = pi*r.^2;
C = 2*pi*r;
```

예제 : Circle

만일 반경이 4라면, 이 함수는 다음과 같이 호출된다.

```
>> [A, C] = circle(4)
A =
   50.2655
C =
   25.1327
```

입력 변수도 출력 변수도 없는 예-1

함수는 입력 변수도 없고, 출력 변수도 없을 수도 있다.

```
function hello
disp('Hello, MATLAB!')
end
```

```
function hello()
disp('Hello, MATLAB!')
end
```

입력 변수도 출력 변수도 없는 예-2

함수는 입력 변수도 없고, 출력 변수도 없을 수도 있다.

```
function [] = hello
disp('Hello, MATLAB!')
end
```

```
function [] = hello()
disp('Hello, MATLAB!')
end
```

위의 4가지 모두 hello 또는 hello() 으로 호출하면 'Hello, MATLAB!'가 출력된다.

함수 정의문의 예

1. 하나의 입력, 하나의 출력:

```
function [area_square] = square(side)
```

2. 하나의 입력, 하나의 출력에 대하여 대괄호는 선택적이다:

```
function area_square = square(side)
```

3. 세 개의 입력, 하나의 출력:

```
function [volume_box] = box(height,width,length)
```

함수 정의문의 예

4. 하나의 입력, 두 개의 출력:

```
function [area_circle,circumf] = circle(radius)
```

5. 출력 변수가 없는 함수:

```
function sqplot(side)
```

함수 예-1

```
function [dist,vel] = drop(g,v0,t)
% Compute distance and velocity
% for a falling object
vel = g*t + v0;
dist = 0.5*g*t.^2 + v0*t;
```

함수 예-2

1. 함수 정의에 사용된 변수 이름은 함수가 호출될 때 사용될 수 있지만, 꼭 사용되어야 할 필요는 없다.

```
>> a = 32.2;
>> initial_speed = 10;
>> time = 5;
>> [feet_dropped,speed] = ...
   drop(a,initial_speed,time)
```

함수 예-3

2. 함수를 호출하기 전에 함수 밖에서 입력 변수에 값을 지정하지 않아도 된다.

```
>> [feet_dropped,speed] = drop(32.2,10,5)
```

3. 입력과 출력은 배열이 되어도 된다.

```
>> [feet_dropped,speed] = drop(32.2,10,0:1:5)
```

이 함수를 호출하면 배열 feet_dropped와 speed를 만들고, 각각은 배열 time에 있는 여섯 개의 시간값에 대응하는 여섯 개의 값을 갖는다.

전역 변수-1

- 전역 변수(global variable)는 지역 변수와 달리 MATLAB 프로그램 전체에서 사용할 수 있는 변수이다.
- 전역 변수는 함수 내부와 기본 작업공간에서 공유되는 변수이다.
- 기본 작업공간과 이 변수들을 전역으로 선언한 다른 함수들에서 사용 가능하다
- 다음은 전역 변수를 사용하는 함수 정의의 예이다.

```
function result = distance(t)
global G
result = 1/2*G*t.^2;
end
```

전역 변수-2

- 전역 변수 G가 정의되지 않은 상태에서 함수를 호출하면 결과가 올바르게 계산되지 않는다.
- 예는 다음과 같은 형태이다.

```
>> a = distance(2)
a =
     []
```

전역 변수-3

- 전역 변수 G를 선언하고 값을 할당하면 함수에서 정상적으로 사용할 수 있는 구조이다.
- 예는 다음과 같은 형태이다.

```
>> global G
>> G = 9.8;
>> a = distance(2)
a =
    19.6000
```

함수 핸들-1

- 어떤 함수든지 함수 핸들은 함수 이름 앞에 부호 @를 사용하여 마치 변수처럼 사용가능하다.
- 다음에 함수를 참조하기 위하여 핸들을 사용할 수도 있다.

예를 들어, 함수 $y = x + 2e^{-x} - 3$의 핸들을 생성하기 위하여 다음의 함수 파일을 정의한다.

```
function y = f1(x)
y = x + 2*exp(-x) - 3;
```

그 다음 함수를 입력변수로서 다른 함수로 넘겨줄 수 있다. 예를 들어, 다음과 같이 $0 \le x \le 6$에서 함수의 그래프를 그릴 수 있다.

```
>> fplot(@f1, [0,6])
```

fzero 를 이용한 함수의 영점 구하기

- `fzero` 함수는 x로 표시되는 하나의 변수로 된 함수의 영점을 구하는 함수이다. 즉, f(x)=0 을 만족하는 x값인 x절편을 구해주는 내장함수.

```
fzero(@function, x0)
```

- `@function`은 함수 `function`에 대한 함수 핸들
- `x0`는 사용자가 제공하는 영점에 대한 예측값

예

```
>> fzero(@cos, 2)
```

- 결과는 2 근처의 영점인 1.5708 을 반환
- fzero 는 영점을 하나씩 찾음. 모든 영점을 찾으려면 grid search 등을 이용한 알고리즘 이용. (나중에 배움)

영점 찾기의 가장 안전한 방법

- 가장 안전한 방법은 구간을 사용하는 방식이다.

```
fzero(@f, [a b])
```

- 이 방법은 구간 $[a, b]$를 지정하여 영점을 찾는 방식이다.
- 이때 다음 조건을 만족해야 한다.

$$f(a) \cdot f(b) < 0$$

- 위 조건이 만족되면, 그 구간 안에 영점이 반드시 존재하는 구조이다.
- 따라서 거리와 상관없이 해당 구간의 영점을 안정적으로 찾는 방법이다.
- 영점이 해당 구간에 여러 개 있을 경우 fzero는 하나만 찾음. 어떤 걸 찾을지는 초기 내부 과정에 따라 달라짐

사용자 정의함수와 fzero의 사용-1

fzero 함수를 이용하여 좀 더 복잡한 함수의 영점들을 구하고자 한다면, 함수 파일 안에 함수를 정의하는 것이 더 편리하다.

예를 들어, 다음의 함수 파일을 정의한다.

```
function y = f1(x)
y = x + 2*exp(-x) - 3;
```

즉,

$$y = x + 2e^{-x} - 3$$

을 고려한다.

사용자 정의함수와 fzero의 사용-2

- 미리 어느 정도 x 절편의 위치를 근사적으로 알고 시작.
- 함수의 그래프를 그리면, $x = -0.5$ 근처에 하나의 영점과 $x = 3$ 근처에 하나의 영점이 있는 것을 보인다.
- $x = -0.5$ 근처의 영점의 좀더 정확한 값을 구하려면, 다음을 입력한다.

```
>> x = fzero(@f1,-0.5)
x =
   -0.5881
```

함수의 최소값 구하기-1

fminbnd 함수는, x로 나타내지는, 하나의 변수를 갖고 있는 함수의 최소값을 찾는다. 기본 구문은 다음과 같다.

```
fminbnd(@function, x1, x2)
```

여기에서 @function은 함수에 대한 함수 핸들이다. fminbnd 함수는 구간 $x_1 \leq x \leq x_2$에서 함수를 최소로 만드는 값을 반환한다.

예를 들어,

```
>> fminbnd(@cos,0,4)
```

는 값 3.1416을 반환한다.

함수의 최소값 구하기-2

fminbnd를 사용할 때, 함수 파일 안에서 함수를 정의하는 것이 편리하다. 예를 들어, 다음의 함수 파일을 정의한다.

```
function y = f2(x)
y = 1 - x.*exp(-x);
```

$0 \leq x \leq 5$에서 y의 최소값을 주는 x의 값을 구하기 위해서는, 다음을 입력한다.

```
>> x = fminbnd(@f2,0,5)
x =
    1.0000
```

그리고 y의 최소값을 구하려면 `y = f2(x)`를 입력한다. 결과는 $y = 0.6321$이다.

함수의 최소값 구하기-3

- 함수는 하나 이상의 로컬 최소값과 하나의 글로벌 최소값을 가질 수 있다.
- 독립 변수의 지정된 범위가 글로벌 최소값을 포함하고 있지 않으면, `fminbnd`는 글로벌 최소값을 찾지 않는다.
- `fminbnd`는 경계에서 발생하는 최소값도 구한다.

함수의 최소값 구하기-4

하나 이상의 변수를 갖는 함수의 최소값을 구하기 위하여 `fminsearch` 함수를 이용한다. 기본 구문은 다음과 같다.

```
fminsearch(@function, x0)
```

여기에서 `@function`은 궁금해 하는 함수의 함수 핸들이다. 벡터 `x0`는 사용자에 의하여 반드시 제공되어야 하는 예측값이다.

함수의 최소값 구하기-5

함수를 최소화하기 위하여, 원소들이 $x(1) = x$ 및 $x(2) = y$인 벡터를 사용하여 m-파일을 먼저 정의한다.

```
function f = f4(x)
f = x(1).*exp(-x(1).^2 - x(2).^2);
```

최소값이 $x = y = 0$ 근처에 있다고 예측했다고 가정한다. 세션은 다음과 같다.

```
>> fminsearch(@f4,[0,0])
ans =
   -0.7071    0.0000
```

이와 같이 최소값은 $x = -0.7071$, $y = 0$에서 발생한다.

함수 핸들-2

함수를 호출하는 한 가지 방법이 함수 핸들 @를 이용하는 것이다.
사용자 정의 함수 fun1과 fzero 함수를 사용하는 것을 고려한다.

```
>> [x, value] = fzero(@fun1,[0, 3])
```

예를 들어,

$$y = x^2 - 4$$

와 같은 함수의 영점을 구하는 상황을 생각할 수 있다.

익명 (Anonymous) 함수-1

익명 함수는 M-파일을 생성할 필요 없이 간단한 함수를 생성할 수 있게 해준다. MATLAB 명령 라인에서 또는 다른 함수나 스크립트 안에서 익명 함수를 만들 수 있다.

식으로부터 익명 함수를 생성하는 구문은 다음과 같다.

```
fhandle = @(arglist) expr
```

여기에서 `arglist`는 함수로 넘겨지는 콤마로 구분된 입력 변수의 목록이며, `expr`은 하나의 유효한 MATLAB 식이다.

익명 함수-2

어떤 수의 제곱을 계산하기 위해 sq라고 부르는 간단한 함수를 생성하기 위하여, 다음을 입력한다.

```
>> sq = @(x) x.^2;
```

가독성을 높이기 위하여, sq = @(x)(x.^2)처럼 식을 괄호로 둘러싸도 좋다. 함수를 실행하기 위해 함수 핸들의 이름을 입력하고, 다음으로 소괄호에 넣은 입력 변수들을 입력한다. 예를 들면

```
>> sq([5,7])
ans =
    25    49
```

익명 함수-3

- `sq([5,7])`을 입력하는 것은 `[5,7].^2`를 입력하는 것보다 한 번 더 키를 쳐야 하기 때문에, 이 특별한 익명 함수가 당신의 작업을 덜어주지 않을 것이라고 생각할 수 있다.
- 그러나 익명 함수는 배열지수에 필요한 점 `.`을 입력하는 것을 잊는 것으로부터 자유롭게 해준다.
- 또한 익명 함수는 많은 키를 입력해야 하는 더 복잡한 함수에 유용하다.

익명 함수-4

익명 함수의 핸들은 다른 함수에 전달할 수 있다. 예를 들어, 구간 $[-10, 10]$에서 다항식의 최소값을 구하기 위하여, 다음을 입력한다.

```
>> poly1 = @(x) 4*x.^2 - 50*x + 5;
>> fminbnd(poly1, -10, 10)
ans =
    6.2500
```

다항식을 다시 사용하지 않을 것이라면, 핸들 정의문을 생략하고 대신 다음을 입력할 수 있다.

```
>> fminbnd(@(x) 4*x.^2 - 50*x + 5, -10, 10)
```

다수의 입력 변수들-1

하나 이상의 입력을 가지는 익명 함수를 생성할 수 있다. 예를 들어 함수

$$\sqrt{x^2 + y^2}$$

를 정의하기 위하여 다음을 입력한다.

```
>> sqrtsum = @(x,y) sqrt(x.^2 + y.^2);
>> sqrtsum(3,4)
ans =
     5
```

다수의 입력 변수들-2

또 다른 예로서, 평면을 정의하는 함수

$$z = Ax + By$$

를 고려한다. 함수 핸들을 생성하기 전에 스칼라 변수 A와 B의 값이 할당되어야 한다. 예를 들어,

```
>> A = 6; B = 4;
>> plane = @(x,y) A*x + B*y;
>> z = plane(2,8)
z =
    44
```

함수 안에서 다른 함수의 호출

익명 함수는 합성 함수를 구현하기 위하여 다른 함수를 호출할 수 있다. 다음 세션에서, 핸들이 h인 함수는 핸들이 f와 g인 함수들을 호출한다.

```
>> f = @(x) x.^3;
>> g = @(x) 5*sin(x);
>> h = @(x) g(f(x));
>> h(2)
ans =
    4.9468
```

즉,

$$h(x) = g(f(x)) = 5\sin(x^3)$$

이다.

변수와 익명 함수-1

- 변수는 익명 함수에서 두 가지 방법으로 나타난다.
- 첫째, 아래와 같이 입력 변수 목록에서 지정된 변수로서 나타난다

```
f = @(x) x.^3
```

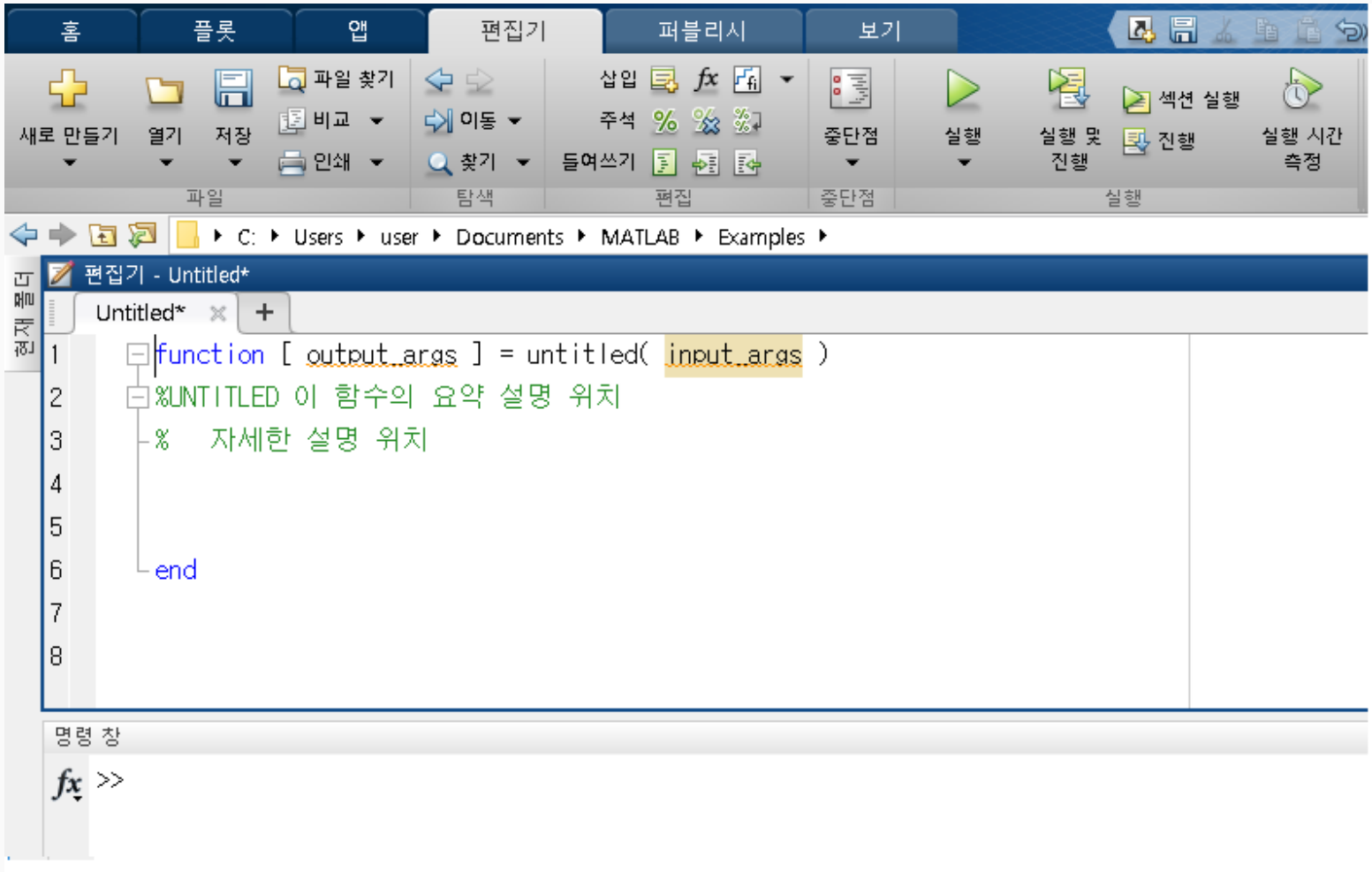
- 둘째, 다음 슬라이드의 예처럼, 식에서 지정된 변수로서도 나타난다.

변수와 익명 함수-2

```
plane = @(x,y) A*x + B*y
```

- A와 B를 갖는 예에서와 같이, 식에서 지정된 변수로서 나타날 수 있다.
- 함수가 만들어질 때, MATLAB은 이 변수값을 수집하고 함수 핸들이 살아있는 동안 이 값들을 유지한다.
- 이 예에서, 핸들이 생성된 후에 A 또는 B의 값이 변해도, 핸들과 관련된 값들은 변하지 않는다.
- 이러한 특징은 장점과 단점을 모두 갖고 있으며, 항상 이러한 것을 염두에 두고 있어야 한다.

새로운 함수를 생성할 때의 디폴트 편집기 창-2



스프레드시트 파일 가져오기

일부 스프레드시트 프로그램은 데이터를 `.wk1` 형식으로 저장한다.

```
M = wk1read('filename')
```

을 이용하여 이 데이터를 MATLAB으로 가져오고 행렬 M에 저장할 수 있다.

```
A = xlsread('filename')
```

는 마이크로소프트 엑셀 작업 파일 `filename.xls`을 배열 A로 가져올 수 있다.

```
[A, B] = xlsread('filename')
```

는 모든 수치 데이터는 배열 A로, 모든 문자 데이터는 셀 배열 B로 가져온다.

수업 실습 예제

1. 사용자 정의 함수 `squareSum(x,y)`라는 이름의 $x^2 + y^2$를 계산하는 함수를 정의하고 x=2, y=3 일 때의 값을 출력하시오.
2. 함수 `poly1(x)=x`$^2 - 5x + 6$ 을 정의하고, 구간 $0 \leq x \leq 5$에서 그래프를 그리시오.
3. `fzero`를 이용하여 방정식 $x^2 - 5x + 6 = 0$의 해를 구하시오.
4. 익명 함수를 이용하여 $f(x) = \sqrt{x^2 + 1}$을 정의하고 값을 구하시오.
5. 함수 $f(x) = x^3 - 3x + 1$의 그래프를 그리고 영점을 구하시오.